

SIMATS ENGINEERING

CO3 ASSESSMENT TOOL 2

NAME: JAIVANT.S

REG NO: 192311326

COURSE NAME: Design and Analysis of Algorithms

COURSE CODE: CSA0613

Q13. Case Study: Digital Dictionary Lookup System (Binary Search)

Introduction

A digital dictionary stores words in **alphabetical (sorted) order**. To find a word quickly, it uses the **Binary Search** algorithm. Binary Search follows the **Divide and Conquer** technique by repeatedly dividing the search space into two halves until the required word is found.

Requirements

- Words must be stored in sorted order.
- Fast retrieval of word meanings.

- Quick response for users.
 - Efficient searching for large datasets.
-

Working of Binary Search

1. Find the middle word.
 2. Compare it with the search word.
 3. If both are equal, display the meaning.
 4. If the search word comes before the middle word, search the left half.
 5. Otherwise, search the right half.
 6. Repeat until the word is found or the search ends.
-

Divide and Conquer Principle

Binary Search divides the sorted list into two equal halves after every comparison. It continues searching only in the half that may contain the required word. This reduces the number of comparisons and improves efficiency.

Example

Dictionary Words:

Apple, Ball, Cat, Dog, Elephant, Fish, Goat, Horse, Lion, Tiger

Search Word: **Horse**

- Middle = Elephant → Search right half.
- Middle = Horse → Word found.

Only **2 comparisons** are needed.

Algorithm (Pseudo Code)

BinarySearch(A, key)

low = 0

high = n - 1

while low <= high

 mid = (low + high) / 2

 if A[mid] == key

 return Found

 else if key < A[mid]

 high = mid - 1

 else

 low = mid + 1

return Not Found

Program:

```
1- def binary_search(words, key):
2-     low, high = 0, len(words) - 1
3-
4-     while low <= high:
5-         mid = (low + high) // 2
6-         if words[mid] == key:
7-             return mid
8-         elif words[mid] < key:
9-             low = mid + 1
10-        else:
11-            high = mid - 1
12-    return -1
13
14 words = ["Apple", "Ball", "Cat", "Dog", "Elephant", "Fish", "Goat",
15          "Horse", "Lion", "Tiger"]
16 key = input("Enter word: ")
17
18 index = binary_search(words, key)
19
20 if index != -1:
21     print("Word Found")
22 else:
23     print("Word Not Found")
```

Output:

```
Enter word: Dog
Word Found

=== Code Execution Successful ===
```

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$
- Space Complexity (Iterative): $O(1)$

Why Binary Search is Optimal

- Searches only half of the remaining words after each comparison.
- Much faster than Linear Search for large dictionaries.
- Requires fewer comparisons.
- Ideal for sorted data such as digital dictionaries.

Conclusion

Binary Search is the best algorithm for a digital dictionary because the words are already sorted alphabetically. By using the Divide and Conquer approach, it provides fast and efficient word lookup with a time complexity of $O(\log n)$.